SYSTEM ARCHITECTURE DOCUMENT

Smart Waste Sorter System

SYSTEM ARCHITECTURE DOCUMENT

Smart Waste Sorter System

Document version: 1.0

Preparation date: 27 October 2026

Prepared by: [Student Name]

Course / class: [Course or Class Name]

Institution: [Institution Name]

Prepared as a preliminary architecture document from the project description available in the conversation. The project ZIP was not provided for code-level verification.

2. Introduction and Project Overview

2.1 Background

The Smart Waste Sorter System is a student project intended to help people identify waste and choose a suitable sorting category. The planned approach uses computer vision so that a camera image can be analyzed by an AI model and the result can be shown to a user.

2.2 Problem Statement

Waste is not always separated correctly because users may be unsure which bin is appropriate. A computer-vision assistant can provide a quick suggestion, although its result depends on the model, lighting, camera quality, and the object being visible.

2.3 Purpose and Objectives

- Recognize waste items using an AI model, reportedly based on YOLO.
- Display a predicted category and confidence information where supported by the implementation.
- Provide a sorting recommendation and voice guidance if the relevant code is enabled.
- Support learning about AI, image processing, software integration, and responsible waste sorting.

2.4 Scope

The known project context includes Python, OpenCV, a YOLO model file in .pt format, and a camera-based workflow. Voice guidance has been discussed as a desired feature. The exact current implementation, supported classes, data storage, and hardware integration have not been inspected because no project ZIP was supplied.

2.5 Intended Users and Benefits

The primary users are students and demonstrators. Potential benefits include faster guidance, a clearer demonstration of AI inference, and a practical way to discuss recycling. The system should be treated as an aid rather than a guarantee that every item is classified correctly.

3. Requirements and Architecture Overview

3.1 Functional Requirements (proposed baseline)

- FR-01: The application can receive an image or camera frame.
- FR-02: The application loads an available detection model and analyzes the input.
- FR-03: The application identifies a predicted class and, where available, its confidence.
- FR-04: The application presents a sorting suggestion for recognized classes.
- FR-05: Voice output, history logging, and hardware actuation are optional until verified in source code.

3.2 Non-Functional Requirements

- Usability: results and instructions should be easy for a beginner to understand.
- Reliability: missing model or camera errors should be handled clearly.
- Performance: inference should run at an acceptable speed on the target computer.
- Maintainability: class mappings, model paths, and thresholds should be documented.
- Privacy: camera frames should not be saved or shared unless necessary and disclosed.

3.3 Expected Environment

Likely software: Python, OpenCV, a YOLO-compatible library, and a .pt model. Exact package versions must be taken from requirements.txt or another project manifest. Likely hardware: a Windows computer and webcam. These details are not confirmed from source files.

3.4 System Boundary

The system boundary includes input capture, model inference, result interpretation, and presentation. Physical sorting equipment, cloud services, external databases, and training pipelines are outside the confirmed baseline unless found in the project.

4. High-Level System Architecture

Figure 1 presents a conceptual architecture based on the project description. It is not a verified source-code map. Optional components are explicitly marked and should be retained in the final report only if they exist in the implementation.

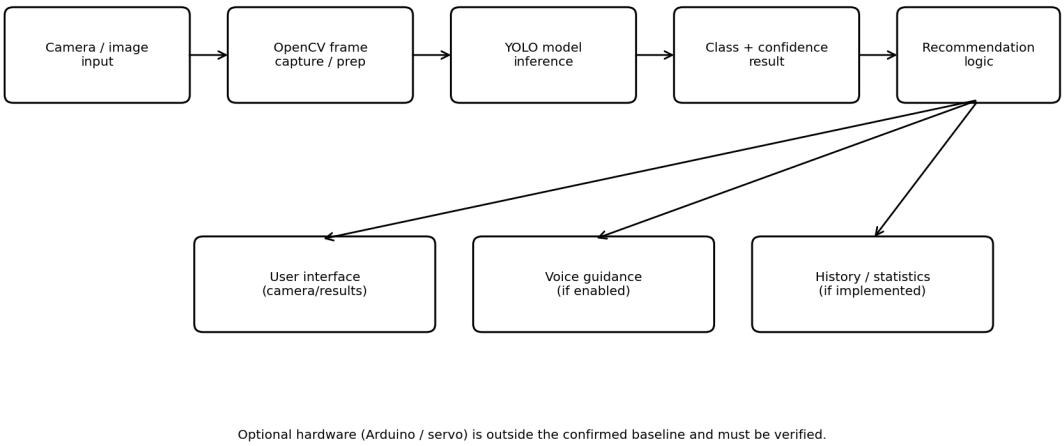


Figure 1. Conceptual high-level architecture; optional components require source-code verification.

4.1 Component Roles

- Input: obtains a camera frame or image.
- OpenCV: commonly handles camera access and frame processing.
- AI inference: uses the loaded YOLO-compatible model to produce predictions.
- Result handling: interprets the predicted class and confidence value.
- Recommendation logic: maps a class to a human-readable sorting suggestion.
- Presentation: displays results and may announce them by voice.
- History or hardware: optional extensions, not confirmed as implemented.

5. Component Architecture and Responsibilities

5.1 Application Entry Point

Project context identifies main.py as the main Python file. Its exact functions and internal structure have not been reviewed. The final implementation-specific document should list the actual function and class names from this file.

5.2 Camera and Image Processing

The camera component obtains frames for analysis. If OpenCV is used as described, the application may use a VideoCapture object and process frames before display or inference. Confirm the camera index, resolution, frame transformations, and release logic from the code.

5.3 AI Model Component

The project context states that YOLO and a .pt model are used or intended. The precise model architecture, package, weights filename, trained classes, and inference parameters remain unverified.

5.4 Recommendation and Presentation

A recommendation component should map model labels to clear disposal guidance. The interface may display a class, confidence score, and recommended bin. Voice output should be documented as implemented only after the relevant code and runtime behavior are checked.

5.5 Optional Logging and Hardware

Statistics, CSV/SQLite history, Arduino serial communication, and servo actuation are possible extensions. No exact storage schema, serial protocol, or hardware component can be confirmed without project files.

5.6 Component Interface Summary

Expected flow: frame → inference result → category/confidence → recommendation → presentation. The real interfaces may be functions, class methods, shared variables, or direct calls in a single Python script; do not assume separate services or modules.

6. AI and Computer Vision Architecture

6.1 Processing Pipeline

A camera-based AI pipeline generally captures a frame, prepares it for inference, sends it to the model, reads predictions, applies any confidence filtering, and displays the result. The actual order and filtering logic must be checked against the source code.

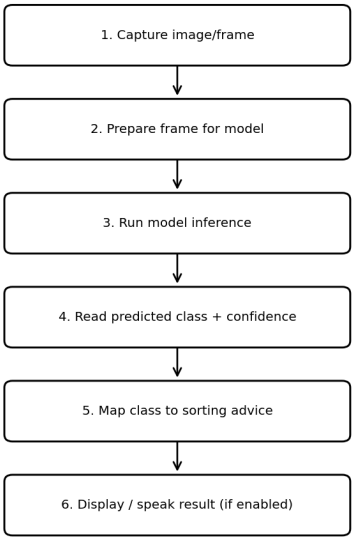


Figure 2. Conceptual data flow from image input to user guidance.

6.2 Model Inference

YOLO models commonly return predicted classes, confidence scores, and bounding boxes for object-detection tasks. The exact output depends on the selected model and how it is called. Do not describe bounding boxes if the project uses a classification-only model.

6.3 Class Mapping and Confidence

The model’s class labels must match the categories used by recommendation logic. A confidence threshold can reduce low-confidence predictions, but an overly strict threshold may cause valid objects to be ignored. Verify the configured threshold and class names in the code or model metadata.

6.4 Model Provenance

The presence of a .pt file does not prove that the student trained the model. Document whether it was downloaded, pretrained, fine-tuned, or trained for this project only when reliable project records support that claim.

6.5 Known Limitations

Potential issues include poor lighting, occlusion, unfamiliar objects, visually similar materials, class imbalance, and camera blur. These are general risks to test, not confirmed failures of the current implementation.

7. Data Flow and Application Workflow

The following diagram describes an expected workflow for a camera-based prototype. It includes a low-confidence branch as a recommended behavior; verify whether the actual code implements it.

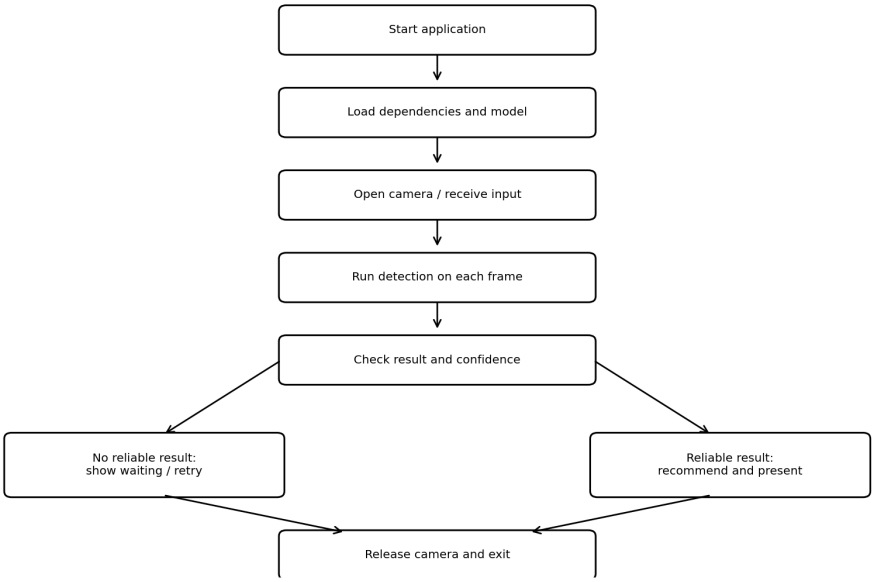


Figure 3. Conceptual application workflow.

7.1 Workflow Description

- Start the application and load required Python packages.
- Load the configured model and initialize the camera or image input.
- Read an input frame and run inference.
- Interpret the model output and check whether a usable prediction exists.
- If no reliable prediction exists, show a waiting or retry message where supported.
- If a reliable prediction exists, map the class to sorting guidance.
- Display the result and optionally provide voice output or record history.
- On exit, release camera resources and close application windows.

7.2 Error and Decision Handling

Recommended checks include a missing model file, unavailable camera, empty prediction, low confidence, and failure to initialize a speech engine. The report cannot claim these checks are already implemented. They should be tested and documented separately.

8. Interface, Integration, and Data Storage

8.1 User Interface

The known context mentions an OpenCV camera window. The precise controls, text labels, window layout, and display logic require inspection. A useful interface should make the predicted category, confidence, and recommended action readable without hiding the camera image.

8.2 Voice Guidance

Voice guidance is a desired project feature and has previously been reported as not speaking when waste is detected. Therefore, this document treats voice output as a feature requiring verification and testing, not as a confirmed working function. Check that speech calls are triggered, do not block the camera loop, and do not repeat continuously for every frame.

8.3 Dependencies and Integrations

OpenCV and a YOLO-compatible inference package are expected from the project context. The exact package versions and additional speech libraries must be verified using requirements.txt and imports in the source code.

8.4 Storage and Statistics

No database or history format is confirmed. If the project records detections, document the real fields, such as timestamp, category, confidence, and recommendation, based on the actual CSV or database implementation.

8.5 Hardware Integration

Arduino or servo control is optional and not confirmed as part of the current software. If present, the final architecture should identify the connection type, message format, actuator positions, safe default state, and failure behavior.

9. Security, Reliability, and Validation

9.1 Reliability Considerations

- Check that model and configuration paths exist before loading.
- Handle camera initialization failure and release the camera on exit.
- Avoid crashing when inference returns no detections.
- Prevent repeated voice announcements for every camera frame.
- Handle file-writing, serial, or speech errors without freezing the interface.

9.2 Privacy and Security

Camera images may include people or private surroundings. Prefer local processing and avoid saving images unless required. Do not include API keys, passwords, or private machine paths in the report or source-code screenshots.

9.3 Architecture Validation Checklist

- [ ] Application starts with the documented command.
- [ ] Model file loads successfully.
- [ ] Camera opens and closes safely.
- [ ] A known sample item produces an expected class.
- [ ] Low-confidence or no-detection cases are handled clearly.
- [ ] Recommendation matches the predicted class.
- [ ] Voice guidance is tested separately if included.
- [ ] History or hardware behavior is tested only if implemented.

9.4 Test Scenarios

Test with representative items from each supported category, different backgrounds and lighting, no item in view, partially hidden objects, and a missing model file. Record actual expected and observed results during testing. No test results are claimed in this preliminary document.

10. Deployment, Conclusion, and Future Improvements

10.1 Deployment

A typical local setup requires a compatible Python installation, project dependencies, model weights, and a camera. Install dependencies from the project's requirements file if available, then run the verified entry-point command. Because the ZIP was not provided, exact installation commands and package versions cannot be confirmed here.

10.2 Packaging

Packaging as a Windows executable with a tool such as PyInstaller may be considered later. Model weights, package data, camera access, and speech dependencies must be included and tested. Executable packaging is a future deployment option unless an existing build configuration is found.

10.3 Future Improvements

- Document and test the supported classes and dataset provenance.
- Improve confidence stabilization and reduce duplicate announcements.
- Add clear recovery messages for camera, model, and speech errors.
- Add structured detection history if required by project scope.
- Integrate sorting hardware only with safe actuator behavior and verified communication.
- Create repeatable evaluation tests and report measured accuracy and latency.

10.4 Conclusion

The proposed architecture separates input capture, AI inference, result interpretation, and user guidance. This structure gives a clear way to explain and test the Smart Waste Sorter System. However, this is a preliminary architecture based on the project description rather than a code audit. Before submission, compare every component, file name, dependency, diagram, and feature status with the actual source code.

Glossary

AI: artificial intelligence. Computer vision: processing and interpreting images. OpenCV: a computer-vision library. YOLO: a family of object-detection models. Inference: using a trained model to make predictions. Confidence score: the model's score for a prediction. Model weights: learned parameters stored in a file such as .pt.

Project References to Verify

Project main file: main.py (reported in project context; not inspected here). Model file: .pt weights file (exact filename unverified). Dependency manifest: requirements.txt (availability and contents unverified). Additional source files, dataset records, and deployment files should be added after inspecting the actual project.